Making a neural network "forget" data: why a common method breaks down

Short report

Khalil Bibi
CentraleSupélec

13 October 2026

In short.

- I studied how to make a small image-recognition neural network (Fashion-MNIST) "forget" part of its training data.
- A common method, NegGrad+, **collapsed** (accuracy at chance level) in 26 out of 55 runs, and only when the data to forget all came from the same class.
- Follow-up experiments show that the problem is linked to how the normalization layers (*BatchNorm*) handle these batches of near-identical examples. Two simple fixes avoided the blow-up in all 30 concentrated cases tested.
- These results are about **optimization stability**. They do not show that the data were actually erased.

1 The problem

A machine-learning model retains something from every example it was trained on. If some data must be removed (for instance at a person's request), the safest solution is to **retrain the model without those data**. This is often too expensive. *Machine unlearning* therefore looks for faster methods that modify the existing model so that it behaves as if it had never seen the data to forget.

My starting question was simple: **does the composition of the data to forget change the outcome?** Is forgetting 600 images taken at random from all classes different from forgetting 600 images of a single class (for example 600 pictures of trousers), given that the other images of that class remain in the retained data?

2 What I tested

- **Data:** Fashion-MNIST, 60,000 small images of clothing in 10 classes.
- **Model:** a small convolutional network (about 1.7 million parameters) with three BatchNorm layers. It gets about 93% of the test images right.
- **Two kinds of forget sets**, always of 600 images:
  - *random*: 60 images from each class;
  - *concentrated*: 600 images from a single class (one version per class, so 10 versions).
- **Two unlearning methods:**

  - *fine-tuning*: simply keep training on the retained data;
  - *NegGrad+*: keep training on the retained data while "pushing" the model to get the data to forget wrong.

- **Reference:** for each case, a model retrained from scratch without the data to forget.
- **Repetitions:** 5 random seeds, i.e. 55 runs per method (11 forget sets × 5 seeds), on an A100 GPU (Google Colab).

The hypotheses, measures and decision rule of this first study ("v3") were written down and dated **before** the main experiment was launched (file dated 12 October 2026, 22:51 UTC). Everything after Section 3 was decided **after** seeing the results: these are exploratory analyses.

3 What happened

Fine-tuning. No detectable difference between random and concentrated forgetting. The model stayed between 90.0% and 93.2% accuracy in all 55 runs.

NegGrad+. The criterion fixed in advance was met, but for the wrong reason: the model was **broken**. In 26 of 55 runs its accuracy dropped to about 10%, the level of a random guess among 10 classes (Table 1). This happened only with concentrated forget sets, never with the random one.

Table 1: Number of collapsed NegGrad+ runs (accuracy ≤ 20%) out of 5 seeds, per forget set.

Forget set	random	c0	c1	c2	c3	c4	c5	c6	c7	c8	c9
Collapsed	0	0	**5**	**5**	0	**5**	**3**	**5**	**3**	0	0

I therefore treated this result as a **failure of the method** in this setting, not as evidence of effective forgetting. Part of the explanation also lies in the tuning: for six classes, the hyperparameter selection procedure picked settings that already broke the model.

4 Why? Three diagnostics

A word on BatchNorm. A BatchNorm layer normalizes the network's internal values using the mean and variance of the current *batch* of images. It also keeps running averages, which are used at test time. In NegGrad+ as I had implemented it, the images to forget went through the network **on their own**. When they all come from the same class, these statistics are very unusual.

Diagnostic 1: running averages or weights? On the 26 collapsed runs, I kept exactly the same weights and changed only BatchNorm's running averages. Median accuracy goes from 10% to 69–74%. The averages therefore play a large role in the measurement, but the weights themselves are damaged (the starting model is at 93%). A control in which the images to forget go through the network but do not count in the loss never collapsed (0 of 55). Merely passing these images through the network is therefore not enough to break the model.

Diagnostic 2: changing how the images go through the network. I compared three versions, on the same batches of images:

- **A**, the original version (images to forget passed on their own);
- **B**, retained images and images to forget passed *together* in a single batch;

- **C**, BatchNorm frozen (evaluation mode) during unlearning.

On the six classes concerned, A blew up (forget loss above 10) in **29 of 30 cases**, B and C in **none** (0 of 30). B and C kept between 91.8% and 93.3% accuracy.

Diagnostic 3: which statistics to use? On class c1 (which always breaks) and class c8 (which never does), I tested three other ways of normalizing the images to forget:

- with the current running averages (**E1**): no blow-up;
- with the batch statistics, but without letting the gradient flow through them (**E1-detached**): the blow-up is much smaller (maximum of 11 to 34, against 190,000 to 300,000 for the original version), but the threshold is still crossed and the model loses a lot of accuracy;
- with the frozen averages of the starting model (**E1-O**): blow-up, including on the control class c8.

Figure 1 shows these trajectories for one seed.

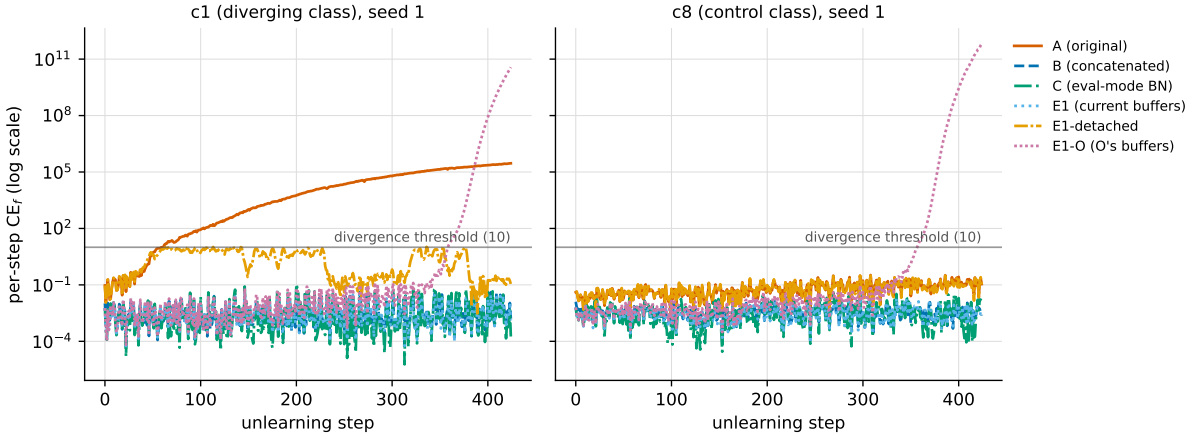


Figure 1: Loss on the images to forget during unlearning (log scale), seed 1, for class c1 (left) and the control class c8 (right). The grey line marks the blow-up threshold (10). The original version A (orange) blows up on c1; B, C and E1 stay stable; E1-O blows up late, even on c8. On c8, the curve of A is hidden under that of E1-detached.

When the blow-up starts. In the original version, the blow-up starts at a different point depending on the class: between the 25th and 37th step (out of 425) for c7, between the 55th and 89th for c1, and after the 180th for c5 and c6 (Figure 2). This reading was made after the fact.

5 What we can say, and what we cannot

We can say: in this small network, NegGrad+ as implemented (images to forget passed on their own, BatchNorm in training mode) becomes unstable when all the images to forget come from the same class. Passing the images together (B) or freezing BatchNorm (C) avoided the blow-up in every concentrated case tested.

We cannot say:

- that the data were actually erased: I ran no privacy test (such as a membership-inference attack);
- that the stable versions forget well: on the images to forget, they stay closer to the starting model than to the retrained model, much like plain fine-tuning;

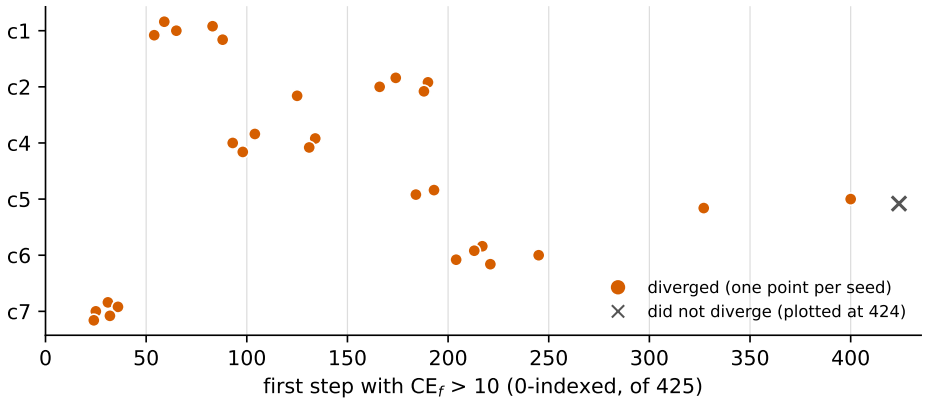


Figure 2: Original version A: first step at which the forget loss exceeds 10, per class (one point per seed; the cross is the only case, c5 seed 4, that never crossed the threshold).

- that the exact mechanism has been identified: these are associations observed under specific interventions, not proof of a general cause.

6 Limitations

- One small network, one dataset, one forget-set size (600), 5 seeds.
- Settings were chosen on a single seed, and the random forget set did not use the same NegGrad+ setting as the concentrated ones.
- The diagnostics were designed after seeing the first results, and the classes studied were chosen accordingly.
- One retrained model per case: the variability of retraining is not measured.
- The thresholds (loss > 10, accuracy $\leq 20\%$) are practical conventions.

Code, results, dated protocols and raw data:
`https://github.com/khalilb1929/machine-Unlearning`.
A more detailed technical manuscript is available in the same repository.